# theFlow!

# Technical Architecture Document

## Visual Node-Based Canvas Application

Version 0.1.0 | June 2026

Author: Xavier Garès | License: GPLv3

# Table of Contents

# 1. Overview

theFlow! is a cross-platform, freeform visual canvas application that allows users to place, connect, and organise heterogeneous media nodes — images, video, audio, documents, text, paint layers, web pages, and PDF files — in an infinite 2D workspace. Nodes are interconnected with Bézier curves, grouped with Backdrops, and annotated with Sticky Notes and canvas paint strokes.

The application is built entirely in Python using the PyQt6 framework, serialises workspaces to a custom JSON-based .flow file format, and is distributed as self-contained native binaries on macOS, Windows, and Linux (DEB and RPM).

## 1.1 Key Capabilities

- Infinite freeform canvas with smooth zoom and pan
- 9 node types: Text, Image, Movie, Audio, Document, Paint, Web, PDF, Dot
- Inline media viewers anchored below nodes (video, audio waveform, PDF, web screenshot)
- Canvas annotation (freehand drawing overlaid on the canvas)
- Bézier connection curves between nodes with socket orientation
- Backdrop grouping with child pinning and recursive refresh
- Full undo/redo stack with per-operation snapshots
- Import/export of node subsets (.flow files)
- Light and dark themes with live settings propagation
- Cross-platform builds: macOS (.app + DMG), Windows (.exe + Inno Setup installer), Linux (.deb, .rpm)
- File association: .flow files open theFlow! on double-click on all platforms

## 1.2 Target Users

| User Type | Primary Use |
|---|---|
| Creative professionals | Mood boards, visual research, project planning |
| Educators | Visual lesson planning, resource organisation |
| Researchers | Reference management, visual note-taking |
| Product managers | Flow diagrams, feature mapping with media references |
| Developers | Architecture diagrams with embedded documentation |

# 2. Technology Stack

## 2.1 Core Language & Runtime

| Component | Version | Role |
|---|---|---|
| Python | 3.9–3.12 | Primary language |
| PyQt6 | 6.4.2+ (Linux/Win), latest (macOS) | UI framework — Qt6 bindings for Python |
| PyQt6-sip | ≥13.4 | C++ ↔ Python binding layer |
| PyQt6-Qt6 | matches PyQt6 | Qt6 shared libraries |

## 2.2 Qt6 Modules Used

| Qt Module | Purpose |
|---|---|
| QtWidgets | QGraphicsScene, QGraphicsView, QDialog, QMainWindow |
| QtCore | QTimer, QPointF, QRectF, QSizeF, QTimeLine, QEvent |
| QtGui | QPainter, QColor, QPen, QBrush, QPixmap, QFont, QSvgRenderer |
| QtMultimedia | QMediaPlayer, QAudioOutput, QAudioDecoder (audio/video) |
| QtMultimediaWidgets | QVideoWidget (video inline viewer) |
| QtSvg / QtSvgWidgets | SVG logo overlay and node badges |
| QtNetwork | Network access for web features |
| QtPrintSupport | Print support |
| QtOpenGL | OpenGL context for multimedia compositing |

## 2.3 Third-Party Dependencies

| Package | Purpose | Platform |
|---|---|---|
| PyInstaller 6.x | Freezes Python app into standalone binary | All |
| Pillow | Icon format conversion (PNG → ICO/ICNS) | Build time |
| cairosvg | SVG → PNG conversion for icons | macOS build |
| dpkg-dev | Builds .deb packages | Linux |
| rpmbuild | Builds .rpm packages | Linux RPM |
| Inno Setup 6 | Builds Windows .exe installer | Windows |

# 3. Application Architecture

## 3.1 Module Structure

theFlow! is structured as a collection of flat Python modules — no package hierarchy. All modules live in the project root directory alongside the entry point.

| Module | Responsibility |
|---|---|
| main.py | Entry point. QApplication subclass, settings loading, file-open event handling |
| view_logic.py | QGraphicsView subclass. Pan/zoom, keyboard shortcuts, context menu, node creation, file I/O |
| scene_logic.py | QGraphicsScene subclass. Serialisation (to_dict/from_dict), undo/redo stack, backdrop containment |
| node.py | All node types: Node, ImageNode, MovieNode, AudioNode, DocumentNode, PaintNode, Dot — plus inline viewers |
| curve.py | ConnectionLine (Bézier), Socket |
| backdrop.py | Backdrop with child pinning and recursive curve refresh |
| note.py | StickyNote, ResizeHandle |
| paint.py | PaintNode with QGraphicsPixmapItem canvas |
| paint_on_canvas.py | CanvasPainter, CanvasStroke, StrokeToolbar (freehand annotation) |
| menu.py | Right-click context menu builder |
| ui_components.py | Tab creation menu (QMenu) |
| settings.py | SettingsDialog with tabbed configuration panels |
| config.py | Global constants, file paths, colour palettes, platform detection |
| utils.py | Shared helpers: bezier math, RichTextEditDialog, menu styling, drag state |
| logo.py | LogoOverlay — animated SVG fade on empty canvas |
| group.py | GroupNode (parked — not yet active) |
| reference.py | ReferenceNode (parked — not yet active) |

## 3.2 Application Entry Flow

On launch, main.py executes the following sequence:

1. Set Qt environment variables (audio backend, OpenGL mode for frozen binary)
2. Instantiate TheFlowApp (QApplication subclass with QFileOpenEvent support)
3. Load or create settings.json from the platform-appropriate config directory
4. Import View (QGraphicsView subclass) — deferred to avoid Qt init issues
5. Construct View, apply settings, show window
6. If sys.argv[1] exists or a QFileOpenEvent was received: open the .flow file after 400ms delay
7. Enter Qt event loop

## 3.3 Settings System

Settings are persisted as JSON to a platform-specific location:

| Platform | Settings Path |
|---|---|
| macOS (frozen) | Next to theFlow.app/Contents/MacOS/theFlow |
| macOS (dev) | Project root /settings/settings.json |
| Windows (frozen) | Next to theFlow.exe |
| Linux (frozen) | ~/.config/theflow/settings.json (XDG) |
| Linux (dev) | Project root /settings/settings.json |

On startup, settings are loaded and merged with DEFAULTS so new keys introduced by app updates are always present. Settings are applied live — changing a setting in the dialog immediately propagates to all existing canvas items via update_from_settings().

# 4. Canvas & Scene System

## 4.1 QGraphicsScene / QGraphicsView

The canvas is built on Qt's QGraphicsScene + QGraphicsView model:

- Scene (scene_logic.py / FlowScene): holds all items, manages serialisation and undo stack
- View (view_logic.py / View): handles viewport transformation, input events, drag-and-drop

The scene uses an infinite coordinate system. The view applies smooth scaling (Ctrl+scroll or pinch) and panning (middle mouse or space+drag). The view's transform matrix maps scene coordinates to viewport pixels.

## 4.2 Coordinate System

All node positions are stored in scene coordinates. When inline viewers (media players, PDF views) are attached below nodes, they are parented to the viewport widget — not the scene — and their pixel position is recalculated on every scroll, zoom, or node move via reposition(). This avoids Qt's limitation that QWidget children of QGraphicsItems don't scale with the scene transform.

## 4.3 Z-Order

| Layer | Z-Value | Items |
|---|---|---|
| Background | 0 | Backdrop |
| Nodes | 10 | All node types, Dot |
| Sticky Notes | 15 | StickyNote |
| Curves | 5 | ConnectionLine |
| Handles | 30 | ResizeHandle, BackdropResizeHandle |
| Temp line | 100 | Dragging connection preview |
| Annotation strokes | 200 | CanvasStroke |

## 4.4 Undo / Redo

The undo stack (scene._undo_stack, scene._redo_stack) stores operation dicts. Each operation type has a corresponding handler in scene_logic._undo_one() and _redo_one(). Supported operation types include:

- add / delete — node/item creation and removal
- move — position change
- resize — size change for Backdrop and StickyNote
- connect / disconnect — curve creation and removal
- backdrop_label / backdrop_color / backdrop_font_size / backdrop_font_color
- sticky_text / sticky_color / sticky_font_size / sticky_font_color
- node_label / node_color / node_font_size / node_font_color / node_shape
- add_stroke / del_stroke / clear_strokes — canvas annotation
- pdf_node_edit / web_node_edit — media node URL/path changes

# 5. Node System

## 5.1 Base Node (node.py)

All content nodes inherit from Node (QGraphicsItem). The base class provides:

- Title (NodeTitle — QGraphicsTextItem): editable label shown on node face
- Input socket (Socket — QGraphicsEllipseItem): left or top, depending on orientation
- Output socket (Socket): right or bottom
- Shape: rectangle, rounded_rectangle, ellipse, diamond, hexagon
- Paint method: selection glow (blue, 10-pass radial gradient), node fill, border, optional icon badge
- Context menu: color, font size, font color, shape, orientation, copy/cut/paste
- Double-click: opens RichTextEditDialog (name + rich text body)
- Inline text viewer: expands below node showing rich text with QTextBrowser (links open in browser)

## 5.2 Node Types

| Type | Shortcut | Key Feature | Inline Viewer |
|---|---|---|---|
| Text Node | T | Rich text with formatting | QTextBrowser with hyperlinks |
| Image Node | I | Loads PNG/JPG/GIF/WebP/SVG | Scaled image with zoom grip |
| Movie Node | M | Loads MP4/MOV/AVI/MKV | QVideoWidget + click-anywhere timeline |
| Audio Node | A | Loads MP3/WAV/AAC/FLAC | RMS waveform + playhead seek + QAudioDecoder |
| Document Node | D | Loads PDF (via QPdfDocument) | QPdfView — scrollable, FitToWidth |
| Paint Node | P | Embedded pixel canvas (QPixmap) | Resizable paint canvas with brushes |
| Dot | Q | Connection pass-through point | None |

## 5.3 Socket & Connection System

Sockets are QGraphicsEllipseItem children of each node. Connections are drawn as QGraphicsPathItem (ConnectionLine) using cubic Bézier curves. Control point direction follows socket orientation (left-right, right-left, top-bottom, bottom-top) allowing connections that visually follow the data flow direction.

Dragging from a socket creates a temporary preview line (magenta). Releasing on a compatible socket creates a persistent ConnectionLine. Multiple outputs can connect to one input; one output can connect to multiple inputs.

## 5.4 Inline Viewer Architecture

Inline viewers are QWidget instances parented to the viewport (not the scene). They anchor below their node and reposition() is called whenever the node moves, the scene scrolls, or the zoom level

changes. Each node type has open_inline_player() / close_inline_player() methods triggered by the ↓ / ↑ keys.

| Viewer Class | Node | Widget Used |
|---|---|---|
| InlineTextViewer | Text Node | QTextBrowser (read-only, hyperlinks) |
| InlineImageViewer | Image Node | QLabel + QScrollArea + resize grip |
| InlineVideoPlayer | Movie Node | QVideoWidget + _ClickSlider timeline |
| InlineAudioPlayer | Audio Node | SimpleWaveformWidget + _ClickSlider |
| InlineDocViewer | Document Node | QPdfView (FitToWidth, scrollable) |

# 6. Audio & Video Pipeline

## 6.1 Audio Node

Audio playback uses Qt's QMediaPlayer + QAudioOutput. The waveform display uses a custom SimpleWaveformWidget that renders an RMS-based gradient silhouette:

- WAV files: synchronous decode via Python wave module
- MP3/AAC/FLAC: asynchronous decode via QAudioDecoder — buffers are accumulated and RMS computed per-frame
- Waveform rendered as a smooth tapered silhouette with a vertical gradient (teal → darker teal)
- Playhead: a vertical line that updates on QMediaPlayer.positionChanged
- Click/drag to seek: custom _ClickSlider on the timeline bar — maps click x-position to media duration

## 6.2 Video Node

Video playback uses QMediaPlayer + QVideoWidget. The same _ClickSlider timeline as audio provides click-anywhere seeking. The inline viewer shows the video at native aspect ratio inside a fixed container.

## 6.3 FFmpeg Backend

Qt6 Multimedia uses the FFmpeg backend on all platforms (bundled inside the PyInstaller binary). GStreamer is NOT used. UPX compression is disabled on Linux builds because it corrupts the bundled FFmpeg shared libraries.

# 7. Serialisation — The .flow File Format

## 8.1 Overview

A .flow file is a UTF-8 JSON file. It is human-readable and version-independent (new keys are ignored by older versions). The root object contains:

- nodes — array of node snapshot dicts
- connections — array of {from_socket, to_socket} string pairs
- strokes — array of canvas annotation stroke dicts (color, width, style, points)

## 8.2 Node Serialisation

Each node type serialises its specific fields. Common fields for all nodes:

| Field | Type | Description |
|---|---|---|
| id | int | Python id() of the item — used as a key for socket mapping |
| type | string | Node type identifier (see below) |
| name | string | Display label |
| x, y | float | Scene position |
| shape | string | rectangle / rounded_rectangle / ellipse / diamond / hexagon |
| color | string | ARGB hex color (#AARRGGBB) |
| color_locked | bool | Whether color was manually set |
| font_size | int | Canvas font size |
| font_color | string | ARGB hex |
| orientation | string | left-right / right-left / top-bottom / bottom-top |

## 8.3 Node Type Identifiers

| Type string | Node class | Extra fields |
|---|---|---|
| node | Node (text) | title_html, body_html |
| image | ImageNode | image_path, image_data (base64 PNG) |
| movie | MovieNode | movie_path |
| audio | AudioNode | audio_path, color (as [r,g,b,a]) |
| doc | DocumentNode | doc_path |
| paint | PaintNode | canvas_data (base64 PNG) |
| dot | Dot | — |
| sticky | StickyNote | sticky_name, sticky_text, color, font_size, font_color, width, height |
| backdrop | Backdrop | backdrop_name, backdrop_text, color, font_size, font_color, width, height |

## 8.4 Connections

Connections are serialised as string pairs using the socket ID map built during to_dict():

```
{ "from": "123456789_out", "to": "987654321_in" }
```

The socket ID format is {node_id}_{in|out}. During from_dict(), a socket_map dict resolves these strings back to live Socket objects.

## 8.5 Import / Export

Export Selected (Ctrl+E) serialises only the selected items and their interconnecting curves, filtering connections whose both sockets are in the selection. Strokes overlapping the selection bounding rect are also included.

Import Nodes (Ctrl+I) calls merge_from_dict() — additive load that does not clear the scene. Imported nodes are offset by 80px to avoid exact overlap with existing content.

# 8. Canvas Annotation

## 9.1 Architecture

Canvas annotation (Ctrl+P) overlays freehand drawing directly on the QGraphicsScene using custom CanvasStroke items (QGraphicsPathItem subclass). The CanvasPainter class manages the draw mode state and intercepts mouse events on the view.

## 9.2 Stroke Persistence

Strokes are serialised in the .flow file as arrays of QPointF coordinates with color, width, and style (solid, dashed, dotted). They are restored on load and participate in the undo stack (add_stroke, del_stroke, clear_strokes operations).

## 9.3 Toolbar

The StrokeToolbar is a floating QWidget shown when annotation mode is active. It provides color picker, stroke thickness slider, line style selector, and a clear-all button. Default values are configurable in Settings → Annotations.

# 9. Build System

## 10.1 Overview

theFlow! is distributed as a self-contained binary on each platform — no Python runtime or dependencies are required on the end-user machine. PyInstaller packages the Python interpreter, all modules, Qt6 shared libraries, and FFmpeg into a single distributable.

## 10.2 macOS Build

| File | Purpose |
|---|---|
| build_mac.sh | Build script — finds Python 3.12, creates venv, installs deps, converts icon, runs PyInstaller, copies icon.icns, code-signs, creates DMG |
| theflow_mac.spec | PyInstaller spec — defines datas, hiddenimports, info_plist with CFBundleDocumentTypes |
| hook_fix_cfbundle.py | Runtime hook — fixes CFBundle reference, installs Qt message handler to prevent abort() on Python exceptions |
| rthook_theflow.py | Runtime hook — sets sys.path and Qt plugin paths from _MEIPASS |
| main.py | Entry point — TheFlowApp subclass with QFileOpenEvent support for Finder double-click |

**Output:** dist/theFlow.app + dist/theFlow-0.1.0.dmg

**Command:** ./build_mac.sh

### macOS File Association

The Info.plist in theflow_mac.spec declares CFBundleDocumentTypes and UTExportedTypeDeclarations for the .flow extension. The build script copies icon.icns into the bundle Resources folder. After first launch, LaunchServices registers the association automatically. Finder double-click sends a QFileOpenEvent which TheFlowApp.event() intercepts and routes to view.open_path().

## 10.3 Windows Build

| File | Purpose |
|---|---|
| build_windows.bat | Build script — runs PyInstaller then Inno Setup |
| theflow_windows.spec | PyInstaller spec — onefile mode, hiddenimports, version_info.txt |
| rthook_windows.py | Runtime hook — sets sys.path and Qt plugin paths |
| main_windows.py | Entry point — no macOS env vars, sys.argv file open with 400ms delay |
| theflow_setup.iss | Inno Setup script — installs exe, registers .flow extension in HKCU registry, creates Start Menu and optional desktop shortcut |
| version_info.txt | EXE metadata — FileVersion, ProductVersion, company info embedded in the .exe |

**Output:** dist\theFlow.exe + dist\theFlow-0.1.0-setup.exe

**Command:** build_windows.bat

### Windows File Association

The Inno Setup script writes to HKCU\Software\Classes\.flow pointing to theFlow.Document. The DefaultIcon entry uses theFlow.exe,0 (first icon embedded in the exe). Double-clicking a .flow file from Windows Explorer passes the path as sys.argv[1] to the running process.

## 10.4 Linux DEB Build

| File | Purpose |
|---|---|
| build_deb.sh | Runs PyInstaller, assembles /tmp/theflow_0.1.0_amd64/ tree, writes DEBIAN/control and postinst, runs dpkg-deb, moves .deb to dist/ |
| theflow_linux.spec | PyInstaller spec for Linux — no macOS hooks, FFmpeg bundled |
| setup_ubuntu.sh | First-time setup: installs Python 3.9, libpython3.9-dev, PyQt6 6.4.2, Playwright |
| theflow.desktop | FreeDesktop .desktop entry with MimeType=application/x-theflow |
| theflow-mime.xml | MIME type declaration for .flow extension |
| main.py (Linux) | Entry point — sets QT_PLUGIN_PATH, LD_LIBRARY_PATH from _MEIPASS when frozen |

**Output:** dist/theflow + dist/theflow_0.1.0_amd64.deb

**Command:** ./build_deb.sh

## 10.5 Linux RPM Build

The RPM build follows the same PyInstaller step but packages via rpmbuild. The build_rpm.sh script:

- Creates the ~/rpmbuild directory tree
- Tarballs dist/theflow as the source
- Copies the desktop file, icon, and manual into SOURCES/
- Copies theflow.spec (RPM spec, not PyInstaller spec) to SPECS/
- Runs rpmbuild -bb
- Copies the resulting .rpm to dist/

**Output:** dist/theflow + dist/theflow-0.1.0-1.x86_64.rpm

**Command:** ./build_rpm.sh

# 10. File Association

## 11.1 Summary

| Platform | Mechanism | Handler |
|---|---|---|
| macOS | CFBundleDocumentTypes + UTExportedTypeDeclarations in Info.plist | QFileOpenEvent → TheFlowApp.event() → view.open_path() |
| Windows | HKCU\Software\Classes\.flow registry key (written by Inno Setup) | sys.argv[1] → view.load_file() with 400ms delay |
| Linux DEB/RPM | MIME type (application/x-theflow) via theflow-mime.xml + .desktop MimeType | sys.argv[1] → view.load_file() with 400ms delay |

## 11.2 Logo Fade on File Open

When the app starts with an empty canvas, the LogoOverlay fades in over 200ms. If a file is opened immediately (double-click from file manager), there is a 400ms delay before open_path()/load_file() is called — this ensures the fade-in animation completes before the fade-out is triggered. Additionally, open_path() forces _opacity = 1.0 and calls _start_fade(fade_in=False) immediately to guarantee a clean fade-out even if timing is off.

# 11. Platform Differences & Known Constraints

| Feature | macOS | Windows | Linux |
|---|---|---|---|
| Python version | 3.12 (venv) | 3.12 (system) | 3.9 (system) |
| PyQt6 version | Latest | Latest | 6.4.2 (pinned) |
| Exit method | os._exit() (prevents destructor race segfault) | sys.exit() | sys.exit() |
| Audio backend | AVFoundation (coreaudio) | Windows Media | FFmpeg (bundled) |
| UPX compression | Ĺ Allowed | Ĺ Allowed | φ Disabled (corrupts FFmpeg libs) |
| Icon format | .icns (iconutil via sips) | .ico (Pillow) | .png (256x256) |
| Settings location | Next to exe / dev root | Next to exe | ~/.config/theflow/ |
| Installer | DMG (drag to Applications) | .exe (Inno Setup) | .deb or .rpm |

# 12. Key Design Decisions

## 13.1 Flat Module Structure

All modules are flat Python files rather than a package. This simplifies PyInstaller bundling (no package __init__.py imports to manage) and keeps import paths short. The tradeoff is that the project root becomes the effective namespace.

## 13.2 Viewport-Parented Inline Viewers

Inline viewers are QWidget instances parented to the viewport, not to scene items. This means they don't automatically scale with the canvas zoom, but it avoids severe Qt limitations: QWidget children of QGraphicsProxyWidget don't support multimedia (QVideoWidget, QPdfView) and have clipping issues. The manual reposition() pattern, while verbose, is reliable across all Qt multimedia widget types.

## 13.3 No WebEngine in Distributed Binary

QtWebEngineCore is not bundled in the distributed binary. Web page screenshots are produced by Playwright/Chromium (a user install) rather than embedding a full Chromium build in the app. This keeps the binary size manageable and avoids macOS AVFoundation compositor conflicts that arise when WebEngine is bundled with PyInstaller.

## 13.4 JSON Serialisation

The .flow format uses plain JSON rather than a binary format or SQLite. This makes files human-readable, diffable in version control, and forward-compatible — future node types can add new keys without breaking old files. Base64-encoded PNG is used for embedded image data (paint canvas, web thumbnails) where binary blobs are unavoidable.

## 13.5 Separate Entry Points per Platform

macOS, Windows, and Linux each have a separate main.py / main_windows.py. This avoids a single entry point cluttered with platform guards and makes each platform's startup sequence easy to read and debug independently.

# 13. Dependency Installation Reference

## macOS (one-time)

```
brew install python@3.12
./build_mac.sh  # handles all other deps in venv
```

## Windows (one-time)

```
pip install pyinstaller PyQt6 PyQt6-Qt6 PyQt6-sip
winget install JRSoftware.InnoSetup
python convert_icon_windows.py   # create icons/logo.ico
build_windows.bat
```

## Linux Ubuntu/Debian (one-time)

```
chmod +x setup_ubuntu.sh && ./setup_ubuntu.sh
./build_deb.sh
```

## Linux Fedora/RHEL (one-time)

```
sudo dnf install python3.9 python3.9-devel rpm-build
pip3.9 install pyinstaller 'PyQt6==6.4.2' 'PyQt6-Qt6==6.4.2' playwright
./build_rpm.sh
```

# 14. Glossary

| Term | Definition |
|---|---|
| .flow file | theFlow! workspace file — JSON containing nodes, connections, and annotation strokes |
| Backdrop | A translucent grouping rectangle that pins child nodes and moves them together |
| CanvasStroke | A freehand annotation path drawn over the canvas (QGraphicsPathItem) |
| ConnectionLine | A Bézier curve connecting two node sockets |
| DXA | Document XML Attributes unit — 1/1440th of an inch, used by Word/docx |
| FFmpeg backend | Qt6 Multimedia's media playback engine (replaces GStreamer on Linux in Qt6) |
| FitToWidth | QPdfView zoom mode — scales each PDF page to fill the viewer width |
| Inline viewer | A QWidget anchored below a node showing media content (video, audio, PDF, web screenshot) |
| _MEIPASS | PyInstaller temp directory where frozen binary extracts bundled files at runtime |
| Node | A rectangular canvas item with an input and output socket |
| Playwright | Microsoft's headless browser automation library — used to screenshot web pages |
| QFileOpenEvent | Qt event sent to the application when macOS asks it to open a file |
| RMS | Root Mean Square — amplitude measure used to compute the audio waveform silhouette |
| Socket | A circular connection point on a node (input or output) |
| StrokeToolbar | Floating toolbar shown during canvas annotation mode |
| UTI | Uniform Type Identifier — macOS system for declaring file types (com.xaviergares.theflow.document) |
| venv | Python virtual environment — isolated Python + packages used for macOS builds |

*End of Document — theFlow! v0.1.0 Technical Architecture*